pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

pandas: Series and DataFrames, Part 1

WEEK 2, DAY 2, PART 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's tenth note

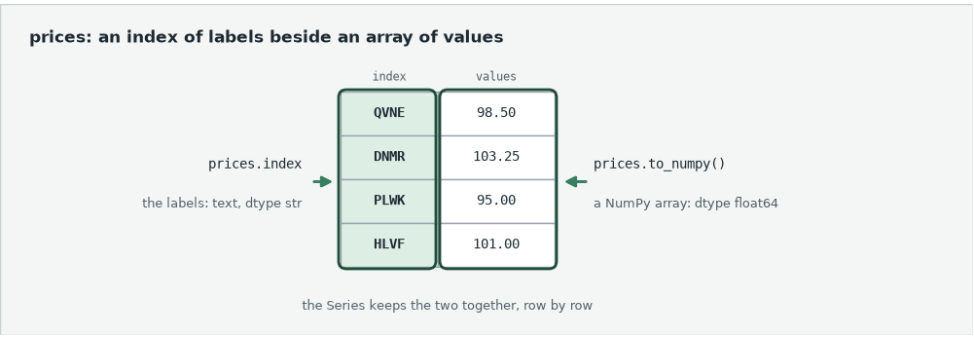
Yesterday's file came back without its tickers. Today the book is two hundred bonds in holdings.csv. First put names on our four bonds, and check the coupon income still ties when the other system sends the faces in its own order, and when it leaves one out. Then load the file: how big, which columns, anything missing. Show me the spreads, and pull up CUSIP 99039NAC0 and its price.

pandas holds a table: text and numbers side by side, a name on every row and column.

A Series is one labelled column; a DataFrame is a table of them.

7.1 A Series: one labelled column

```
import pandas as pd
price_list = [98.5, 103.25, 95.0, 101.0]
print(pd.Series(price_list))
ticker_list = ['QVNE', 'DNMR', 'PLWK', 'HLVF']
prices = pd.Series(price_list, index=ticker_list)
print(prices)
QVNE     98.50
DNMR    103.25
PLWK     95.00
HLVF    101.00
dtype: float64
```



The left column is the index: a label on every row; given none, pandas numbers them from 0.

```
print(prices.to_numpy())
print(type(prices.to_numpy()))
print(prices.index)
[ 98.5  103.25  95.   101.  ]
<class 'numpy.ndarray'>
Index(['QVNE', 'DNMR', 'PLWK', 'HLVF'], dtype='str')
```

The values are a NumPy array, so NumPy's arithmetic still works.

By label or by position

```
print('PLWK:', prices['PLWK'])
print('First price:', prices.iloc[0])
PLWK: 95.0
First price: 98.5
```

```
print(prices[['PLWK', 'QVNE']])
print(prices.iloc[-2:])
PLWK    95.0
QVNE    98.5
dtype: float64
PLWK     95.0
HLVF    101.0
dtype: float64
```

Excel side by side: tickers in A2:A5, prices in B2:B5

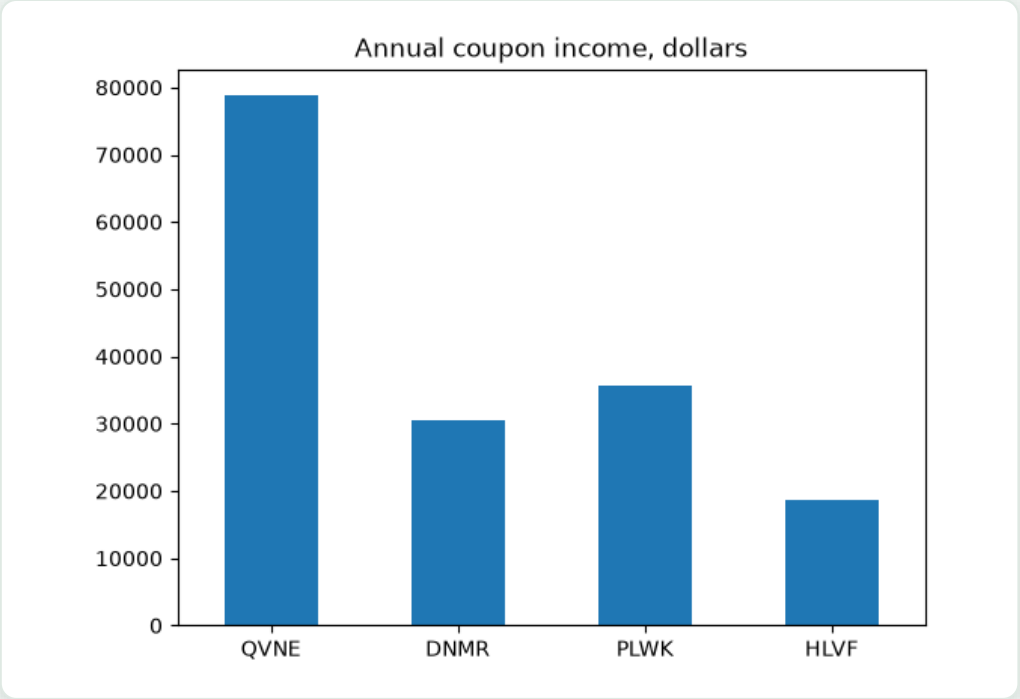
Look up	pandas	In Python, like	Excel
by label	`prices['PLWK']`	a dictionary key	`=VLOOKUP("PLWK", A2:B5, 2, FALSE)`
by position	`prices.iloc[0]`	a list index, from 0	`=INDEX(B2:B5, 1), counting from 1`

An array brings no labels; a dictionary brings its keys.

The book's income, by ticker

```
coupons = pd.Series([5.25, 6.125, 4.75, 7.5], index=ticker_list)
faces = pd.Series([1500000, 500000, 750000, 250000], index=ticker_list)
incomes = faces * coupons / 100
print(incomes)
print('Total annual coupon income:', incomes.sum())
QVNE    78750.0
DNMR    30625.0
PLWK    35625.0
HLVF    18750.0
dtype: float64
Total annual coupon income: 163750.0
```

mean(), min(), and max() work as sum() does (Day 5).



163,750: week 1's number, a ticker on every line.

Predict first: decide, then run it in the notebook

Predict 1

Many older tutorials look a Series up by a plain number. Here the labels are tickers. What does "prices[0]" give? A) 98.5, the first price. B) An error. C) QVNE, the first label.

```
prices[0]
```

Predict 2

How many prices does "prices.loc[:'PLWK']" print? A) Two, QVNE and DNMR: a slice stops before its end. B) Three: QVNE, DNMR, and PLWK.

```
print(prices.loc[:'PLWK'])
```

Predict first: two sources, two Series

Predict 3

The other system sends the faces smallest first: HLVF, DNMR, PLWK, QVNE. "coupons" runs QVNE, DNMR, PLWK, HLVF. What does "faces_by_size * coupons / 100" give? A) The four right incomes: each face meets its own bond's coupon. B) Wrong incomes: the first face meets the first coupon, and so on down. C) An error: the two orders differ.

```
faces_by_size = pd.Series([250000, 500000, 750000, 1500000],
                          index=['HLVF', 'DNMR', 'PLWK', 'QVNE'])
print(faces_by_size * coupons / 100)
```

Predict 4

Now the other system leaves HLVF out. What do HLVF's income and the total show? A) An error, as NumPy 6.3 gave for arrays of different sizes. B) No income for HLVF, and a total of the other three, with no error. C) The full 163,750.

```
faces_missing = faces_by_size.drop('HLVF')
incomes_missing = faces_missing * coupons / 100
print(incomes_missing)
print('Total annual coupon income:', incomes_missing.sum())
```

7.2 A DataFrame: a table of columns

```
desk = pd.DataFrame({
    'Ticker': ['QVNE', 'DNMR', 'PLWK', 'HLVF'],
    'Coupon': [5.25, 6.125, 4.75, 7.5],
    'Maturity': [2033, 2034, 2031, 2032],
    'Price': [98.5, 103.25, 95.0, 101.0],
    'Face': [1500000, 500000, 750000, 250000],
    'Rating': ['BBB', 'BB+', 'A-', 'B+'],
})
desk
```

	Ticker	Coupon	Maturity	Price	Face	Rating
0	QVNE	5.250	2033	98.50	1500000	BBB
1	DNMR	6.125	2034	103.25	500000	BB+
2	PLWK	4.750	2031	95.00	750000	A-
3	HLVF	7.500	2032	101.00	250000	B+




```
print(desk.dtypes)
Coupon      float64
Maturity      int64
Price       float64
Face          int64
Rating          str
dtype: object
(4, 6)
```

```
desk_prices = desk['Price']
print(desk_prices)
print(type(desk_prices))
0     98.50
1    103.25
2     95.00
3    101.00
Name: Price, dtype: float64
<class 'pandas.Series'>
```

Everything in 7.1 works on a column.

Rows by slice, and more ways to build a table

Built from	Code	Row labels
a dictionary of lists (Day 5)	`pd.DataFrame({'Price': price_list})`	0, 1, 2, …
a dictionary of Series	`pd.DataFrame({'Coupon': coupons})`	the Series' labels, matched as in 7.1
a list of rows	`pd.DataFrame(rows, columns=['Ticker', 'Coupon', 'Price'])`	0, 1, 2, …
a 2-D array (NumPy, Part 2)	`pd.DataFrame(array, columns=[...])`	0, 1, 2, …, or index=

```
book = pd.DataFrame({'Coupon': coupons, 'Face': faces})
print(book)
rows = [['QVNE', 5.25, 98.5], ['DNMR', 6.125, 103.25]]
risk = pd.DataFrame(rows, columns=['Ticker', 'Coupon', 'Price'])
print(risk)
      Coupon     Face
QVNE   5.250  1500000
DNMR   6.125   500000
PLWK   4.750   750000
HLVF   7.500   250000
  Ticker  Coupon   Price
0   QVNE   5.250   98.50
1   DNMR   6.125  103.25
```

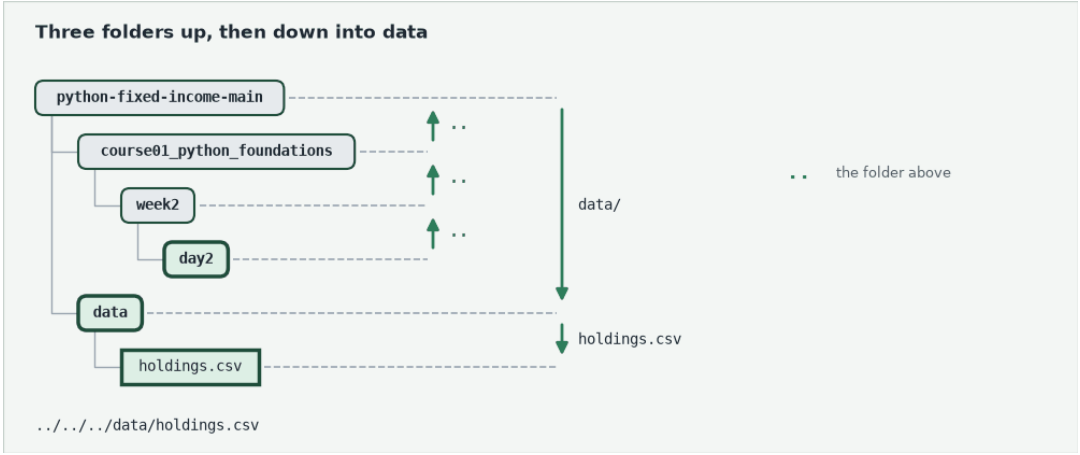
```
print(desk[:2])
  Ticker  Coupon  Maturity   Price     Face Rating
0   QVNE   5.250      2033   98.50  1500000    BBB
1   DNMR   6.125      2034  103.25   500000    BB+
```

The row numbers travel with their rows: reversed, QVNE is still 0.

Nobody types two hundred bonds. Load the file.

7.3 Load the holdings file

```
import os
if os.path.exists('../../../data/holdings.csv'):
    data_path = '../../../data/holdings.csv'
else:
    data_path = 'https://raw.githubusercontent.com/lenamonj/python-fixed-income/main/data/holdings.csv'
holdings = pd.read_csv(data_path)
```




```
print(holdings.shape)
(200, 19)
```

Excel side by side

Ctrl+End lands on S201: the header in row 1, the bonds in rows 2 to 201, where pandas counts 0 to 199.

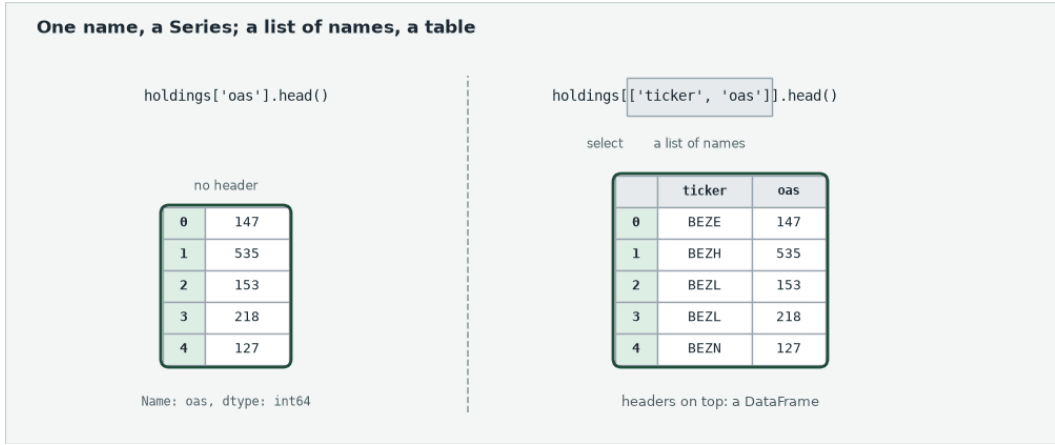
BEZL holds two bonds: a ticker is not a bond.

7.4 One name, or a list of names

```
holdings['oas'].head()
0    147
1    535
2    153
3    218
4    127
Name: oas, dtype: int64
```

```
holdings['OAS']
KeyError: 'OAS'
```

```
holdings[['ticker', 'coupon', 'maturity', 'price']].head()
  ticker  coupon    maturity    price
0   BEZE   7.000  2032-06-18  106.727
1   BEZH   8.125  2029-04-17   97.556
2   BEZL   5.125  2030-06-03   98.730
3   BEZL   6.125  2053-04-09   92.261
4   BEZN   4.625  2036-04-05   92.883
```



Excel side by side

Hiding columns does this on screen; =CHOOSECOLS(A1:S201, 3, 7, 8, 9) in Microsoft 365 returns them by number, where pandas goes by name.

One name, a Series; a list of names, a table.

Predict first: two more

Predict 5

Summarize the coupon, price, yield, spread, and duration columns. A) About the same. B) Above the median. C) Below the median.

```
holdings[['coupon', 'price', 'yield_pct', 'oas', 'duration']].describe()
```

Predict 6

After "by_cusip = holdings.set_index('cusip')", how many columns do "by_cusip" and "holdings" have? A) 19 and 19: the CUSIP is copied into the index. B) 18 and 19. C) 18 and 18.

```
by_cusip = holdings.set_index('cusip')
print(by_cusip.shape, holdings.shape)
by_cusip[['ticker', 'coupon', 'maturity', 'price']].head(3)
```

7.5 Any bond, one CUSIP away

```
print(by_cusip.loc[['99080CAE8', '99039NAC0'], ['ticker', 'price']])
print(holdings.iloc[[0, 2], [2, 8]])
          ticker    price
cusip
99080CAE8   BEZE  106.727
99039NAC0   BEZL   98.730
  ticker    price
0   BEZE  106.727
2   BEZL   98.730
```

```
print(by_cusip.loc['99039NAC0', 'price'])
98.73
by_cusip.loc['99080CAE9']
KeyError: '99080CAE9'
```

```
holdings.iloc[[0, 2], ['ticker', 'price']]
IndexError: .iloc requires numeric indexers, got ['ticker' 'price']
```

Question	pandas	Excel
The third row	`holdings.iloc[2]`	row 4 of the sheet, under the header
The first three bonds, first four columns	`holdings.iloc[0:3, 0:4]`	=TAKE(A2:S201, 3, 4), Microsoft 365
Price of a CUSIP	`by_cusip.loc['99039NAC0', 'price']`	=VLOOKUP("99039NAC0", B:I, 8, FALSE)
The same, without counting columns	`by_cusip.loc['99039NAC0', 'price']`	=INDEX(I:I, MATCH("99039NAC0", B:B, 0))
A CUSIP that is not there	`KeyError`	#N/A

The note's bond: BEZL 5.125% 2030, rated BB, at 98.73.

The note, answered

The note asked	Answer
Names on our four bonds	a Series and a table, a ticker on every row
How big, which columns, anything missing	200 bonds, 19 columns, nothing missing
CUSIP 99039NAC0 and its price	BEZL 5.125% 2030, at 98.73

In Part 2 the table starts answering questions: totals and counts, the bonds that pass a test, the book in order, a what-if on a copy, and a table shown well.